

switch Statement in Java

The `switch` statement is used to evaluate a single variable or expression against multiple possible values, and execute different code depending on which value matches. It is an alternative to using multiple `if-else` statements when you need to check a variable for many possible values.

Syntax:

```
switch (expression) {  
    case value1:  
        // Code to be executed if expression equals value1  
        break;  
    case value2:  
        // Code to be executed if expression equals value2  
        break;  
    case value3:  
        // Code to be executed if expression equals value3  
        break;  
    default:  
        // Code to be executed if expression doesn't match any case  
}  

```

- `expression`: The value or variable that is being tested.
- `case value`: Each `case` represents a possible value that the `expression` might match.
- `break`: The `break` statement is used to exit the `switch` block after a match is found. Without `break`, the program will continue checking all other cases (this is called "fall-through").
- `default`: The `default` case is optional and is executed if none of the `case` values match the `expression`.

Example 1: Basic `switch` statement

```
public class Main {  
    public static void main(String[] args) {  
        int day = 3;  
  
        switch (day) {  
            case 1:  
                System.out.println("Monday");  

```

```

        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    case 4:
        System.out.println("Thursday");
        break;
    case 5:
        System.out.println("Friday");
        break;
    case 6:
        System.out.println("Saturday");
        break;
    case 7:
        System.out.println("Sunday");
        break;
    default:
        System.out.println("Invalid day");
    }
}
}

```

Explanation:

- The variable `day` is set to `3`.
- The `switch` statement checks the value of `day`. When it finds the match `case 3`, it prints `"Wednesday"`.
- The `break` statement ensures that the program exits the `switch` block after printing the result for `case 3`. Without the `break`, the code would "fall-through" to the next case, executing its statements as well.
- If `day` were a value other than 1-7, the `default` case would be executed, printing `"Invalid day"`.

Example 2: Switch statement without `break` (Fall-through)

```

public class Main {
    public static void main(String[] args) {
        int number = 2;

        switch (number) {

```

```

        case 1:
            System.out.println("One");
        case 2:
            System.out.println("Two");
        case 3:
            System.out.println("Three");
        default:
            System.out.println("Invalid number");
    }
}
}

```

Explanation:

- Here, the `switch` statement does not include `break` statements.
- When `number = 2`, the program starts at `case 2` and prints `"Two"`. Since there is no `break`, it continues to the next case and prints `"Three"`, and then it executes the `default` case, printing `"Invalid number"`.
- This is called **fall-through**, where multiple cases are executed until a `break` or the end of the `switch` block is reached.

Example 3: Using default in switch

```

public class Main {
    public static void main(String[] args) {
        int score = 85;

        switch (score) {
            case 90:
                System.out.println("Grade A");
                break;
            case 80:
                System.out.println("Grade B");
                break;
            case 70:
                System.out.println("Grade C");
                break;
            default:
                System.out.println("Grade not available");
        }
    }
}

```

Explanation:

- The `score` is `85`, which does not match any of the `case` values (`90`, `80`, or `70`).
 - Since there is no match, the `default` case is executed, printing `"Grade not available"`.
-

Key Points about `switch` Statement:

1. **Works with Primitives:** The `switch` statement works with primitive data types like `int`, `char`, `byte`, `short`, and `long`. It can also work with `String` (in Java 7 and later).
 2. **No Range Checking:** The `switch` statement checks for exact matches only. If you need to check a range or multiple conditions, an `if-else` statement is more suitable.
 3. **Fall-through Behavior:** If `break` is omitted, the program will continue executing the next case (fall-through), which may not always be desired. Always use `break` unless you specifically want fall-through.
 4. **default Case:** The `default` case is optional but useful to handle situations where none of the cases match the expression.
-